Deep Learning - Assignment 02

Linear Regression & Logistic Regression

Name: Muhammad Anwar Raza

Roll Number: MSDS25002

Course: Deep Learning - Spring 2025

Table of Contents

1. Introduction

2. Task 1: Linear Regression (California Housing)

   2.1 Dataset Description

   2.2 Data Preprocessing

   2.3 Model Architecture

   2.4 Single Run Results

   2.5 Experiment Results

   2.6 Analysis

3. Task 2: Logistic Regression (Titanic)

   3.1 Dataset Description

   3.2 Data Preprocessing

   3.3 Model Architecture

   3.4 Single Run Results

   3.5 Experiment Results

   3.6 Analysis

4. Conclusion

1. Introduction

This report presents the implementation and analysis of two machine learning tasks: Linear Regression using NumPy for house price prediction on the California Housing dataset, and Logistic Regression using NumPy for survival prediction on the Titanic dataset. Both models are trained using Stochastic Gradient Descent (SGD) with mini-batch processing. Multiple experiments are conducted to analyze the effect of hyperparameters such as learning rate, number of epochs, weight initialization, hidden layer size, and normalization on model performance.

2. Task 1: Linear Regression (California Housing)

2.1 Dataset Description

The California Housing dataset contains 16,512 training samples and 4,128 test samples (2,801 after cleaning). Each sample has 8 features: MedInc, HouseAge, AveRooms, AveBedrms, Population, AveOccup, Latitude, and Longitude. The target variable is the median house value for each district (in units of $100,000).

2.2 Data Preprocessing

The test dataset contained NaN values and extreme outliers (e.g., HouseAge of 13 million). The following preprocessing steps were applied:

- Training NaN values filled with column median
- Test rows with NaN values dropped entirely
- Test rows with values far outside the training range removed
- Z-score normalization applied using training data statistics (mean and std)
- Normalized values clipped to [-5, 5] to handle remaining outliers
- Data split into 85% train and 15% validation sets

2.3 Model Architecture

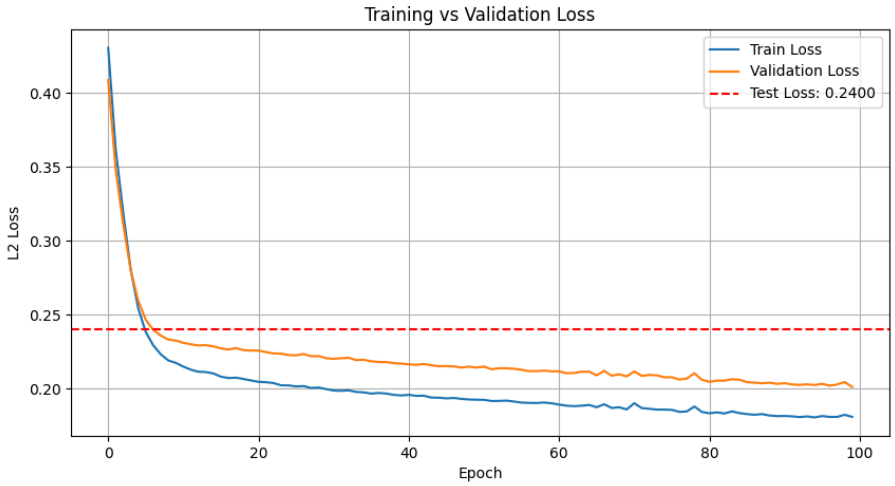
The model is a 2-layer neural network: Input(8) -> Hidden(n) -> Output(1). ReLU activation is used in the hidden layer. The output layer has no activation function (linear output for regression). L2 loss (MSE/2) is used as the loss function. Backpropagation computes gradients through both layers, and SGD updates the weights.

Component	Details
Input Layer	8 features
Hidden Layer	n neurons (8 or 2), ReLU activation
Output Layer	1 neuron, linear (no activation)
Loss Function	L2 Loss = (1/2N) * sum((y - y_hat)^2)
Optimizer	SGD (mini-batch, batch_size=32)
Initialization	He / Random / Zeros

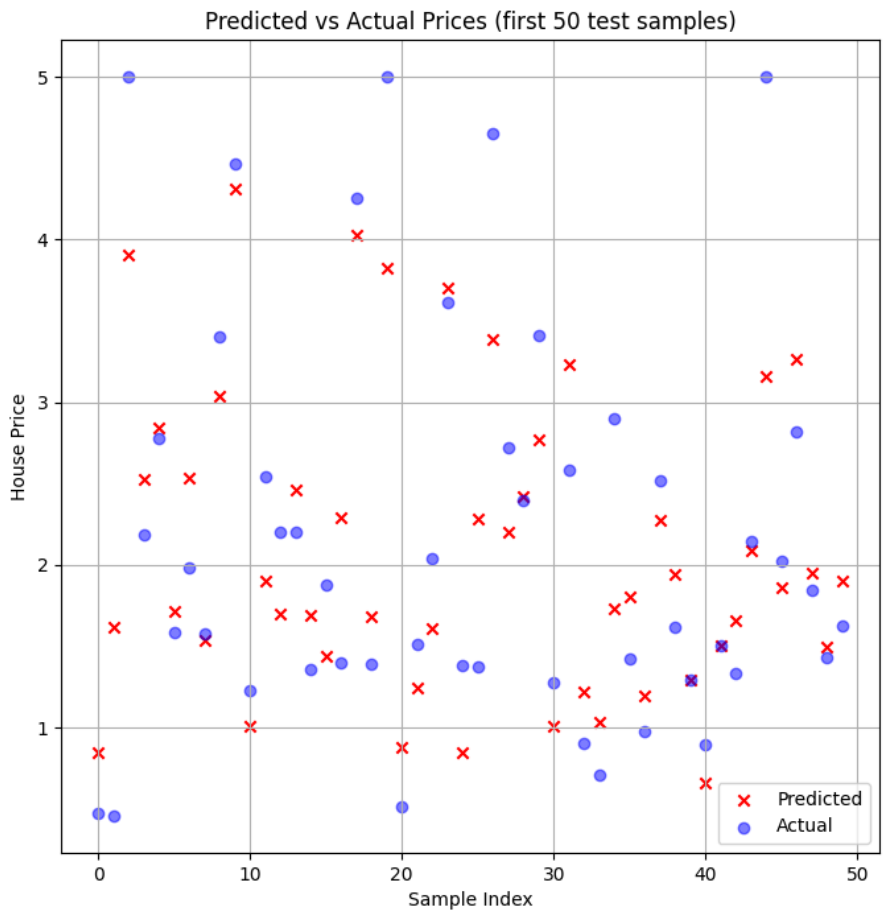
2.4 Single Run Results

The model was trained with: hidden_size=8, init=He, lr=0.01, epochs=100, normalization=True.

Metric	Value
Train R^2 (final)	0.7238
Validation R^2 (final)	0.6951
Test Loss	0.2400
Test R^2	0.6199


Training vs Validation Loss
Train Loss
Validation Loss
Test Loss: 0.2400
L2 Loss
Epoch
0.40
0.35
0.30
0.25
0.20
0
20
40
60
80
100

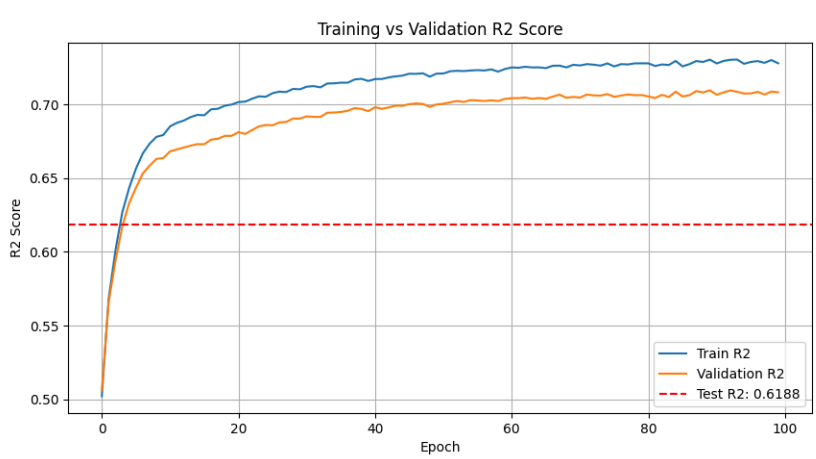
Training vs Validation R2 Score
Train R2
Validation R2
Test R2: 0.6199
R2 Score
Epoch
0.70
0.65
0.60
0.55
0.50
0.45
0.40
0.35
0
20
40
60
80
100


Predicted vs Actual Prices (first 50 test samples)
House Price
Sample Index
Predicted
Actual
0
10
20
30
40
50
1
2
3
4
5

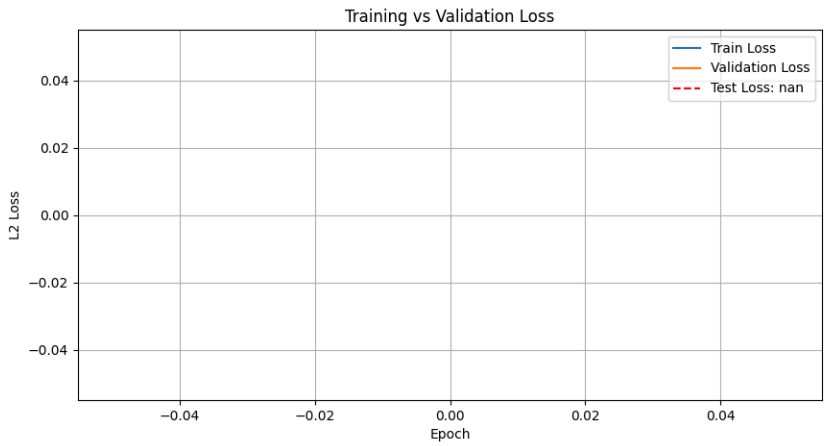
2.5 Experiment Results

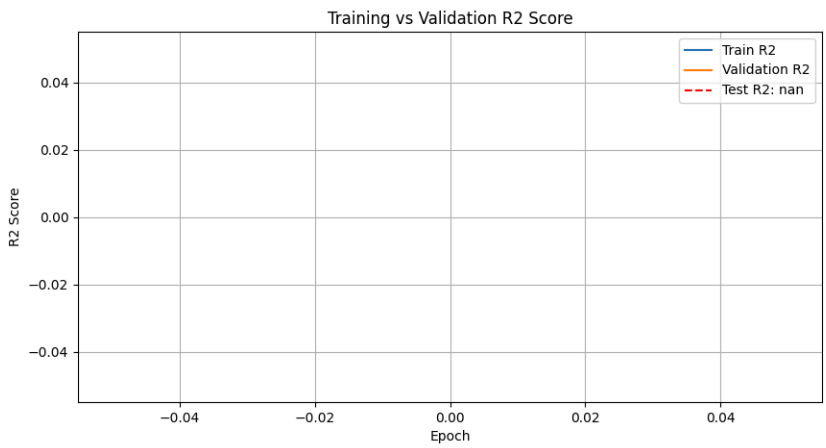
#	Hidden	Init	LR	Epochs	Norm	Test Loss	Test R^2
1	8	He	0.01	100	Yes	0.2406	0.6188
2	8	He	1e-05	100	Yes	0.8207	-0.3000
3	8	He	0.9	100	Yes	NaN	NaN
4	8	He	0.01	150	Yes	0.2390	0.6214
5	8	Random	0.01	100	Yes	0.4037	0.3605
6	8	Zeros	0.01	100	Yes	0.6325	-0.0019
7	2	He	0.01	100	Yes	0.2608	0.5869
8	2	He	0.01	150	Yes	0.2772	0.5609
9	8	He	0.01	100	No	0.6339	-0.0041
10	2	He	0.01	100	No	0.6919	-0.0961

Experiment 1 (Best Config: hidden=8, He, lr=0.01, 100 epochs)

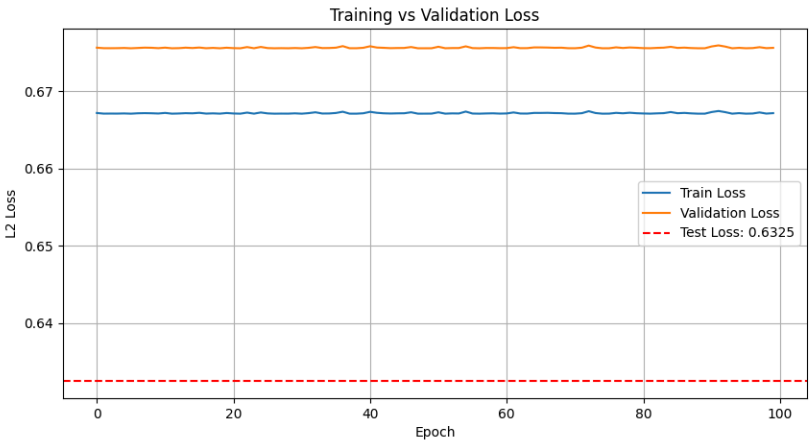


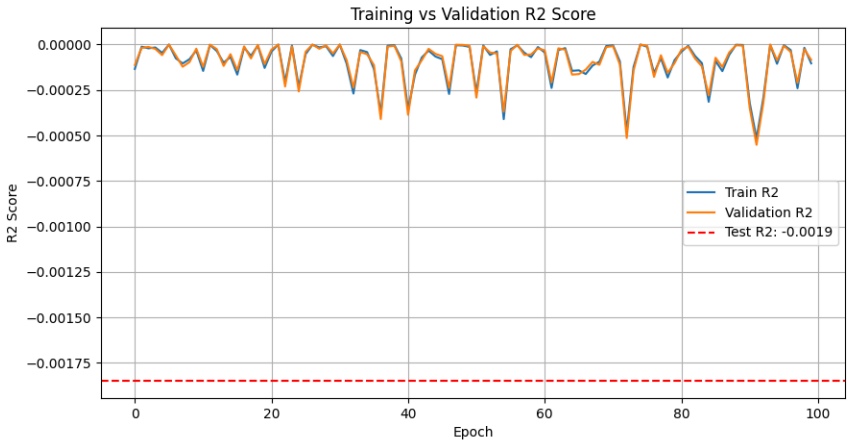
Experiment 3 (lr=0.9 - Exploded)



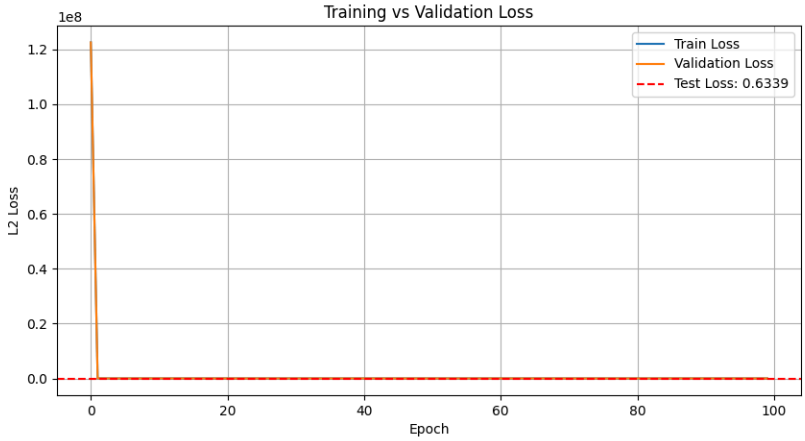


Experiment 6 (Zeros Init - Failed)





Experiment 9 (No Normalization - Failed)


Training vs Validation Loss
1e8
1.2
1.0
0.8
0.6
0.4
0.2
0.0
L2 Loss
0
20
40
60
80
100
Epoch
Train Loss
Validation Loss
Test Loss: 0.6339

Training vs Validation R2 Score
1e8
0.00
−0.25
−0.50
−0.75
−1.00
−1.25
−1.50
−1.75
R2 Score
0
20
40
60
80
100
Epoch
Train R2
Validation R2
Test R2: -0.0041

2.6 Analysis

Best Configuration: Experiment 4 (hidden=8, He init, lr=0.01, 150 epochs, normalized) achieved Test $R^2 = 0.6214$. More epochs helped slightly.

Learning Rate: lr=0.01 worked well. lr=1e-05 was too slow to converge in 100 epochs (Test $R^2 = -0.30$). lr=0.9 caused numerical overflow (NaN values) as gradients exploded.

Weight Initialization: He initialization performed best. Random init worked but slower ($R^2 = 0.36$). Zeros initialization completely failed ($R^2 = -0.002$) because all neurons learn the same thing when starting from zero - this is the symmetry breaking problem.

Hidden Layer Size: 8 hidden neurons ($R^2 = 0.62$) outperformed 2 neurons ($R^2 = 0.59$). More neurons allow the model to learn more complex patterns.

Normalization: Without normalization, the model completely failed (R^2 near 0 or negative). This is because features have very different scales (e.g., Population in thousands vs Latitude around 35). Normalization brings all features to the same scale so gradients are balanced.

3. Task 2: Logistic Regression (Titanic)

3.1 Dataset Description

The Titanic dataset contains 891 training samples and 418 test samples. The goal is to predict whether a passenger survived (1) or died (0). After preprocessing, 9 features are used: Pclass, Sex, Age, SibSp, Parch, Fare, Embarked_C, Embarked_Q, Embarked_S. The dataset is imbalanced: 549 died (61.6%) vs 342 survived (38.4%).

3.2 Data Preprocessing

- Dropped columns: PassengerId, Name, Ticket, Cabin (ID, text, or too many missing values)
- Missing Age values (177 rows) filled with median
- Missing Embarked values (2 rows) filled with mode
- Missing Fare values (1 row in test) filled with median
- Sex converted to numeric: male=0, female=1
- Embarked one-hot encoded into 3 binary columns (C, Q, S)
- Z-score normalization applied with clipping to [-5, 5]
- Data split into 85% train and 15% validation

3.3 Model Architecture

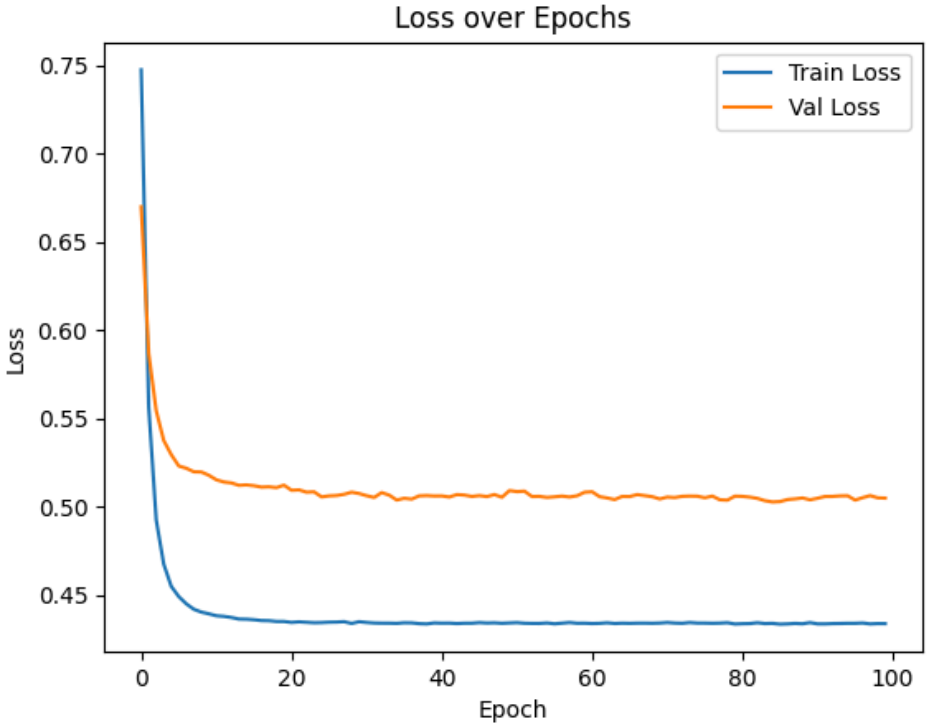
The model is a single-layer logistic regression: Input(9) -> Output(1). Sigmoid activation is used to output a probability between 0 and 1. Binary Cross-Entropy is used as the loss function. If the predicted probability is >= 0.5, the passenger is classified as survived (1), otherwise died (0).

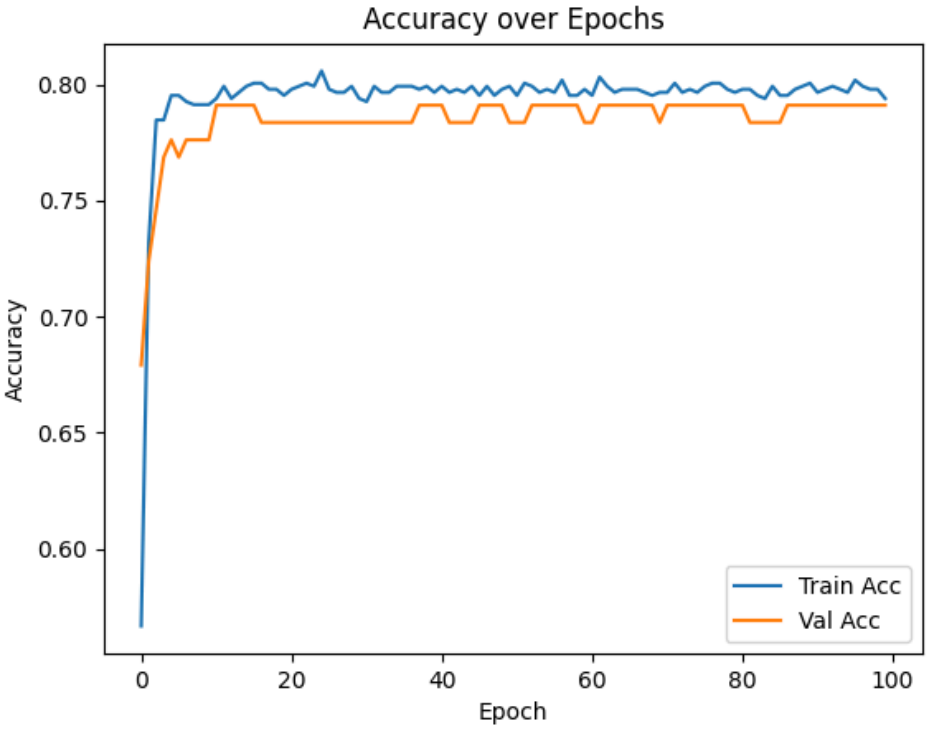
Component	Details
Input Layer	9 features
Output Layer	1 neuron, Sigmoid activation
Loss Function	Binary Cross-Entropy
Optimizer	SGD (mini-batch, batch_size=32)
Decision Threshold	0.5
Initialization	He / Random / Zeros

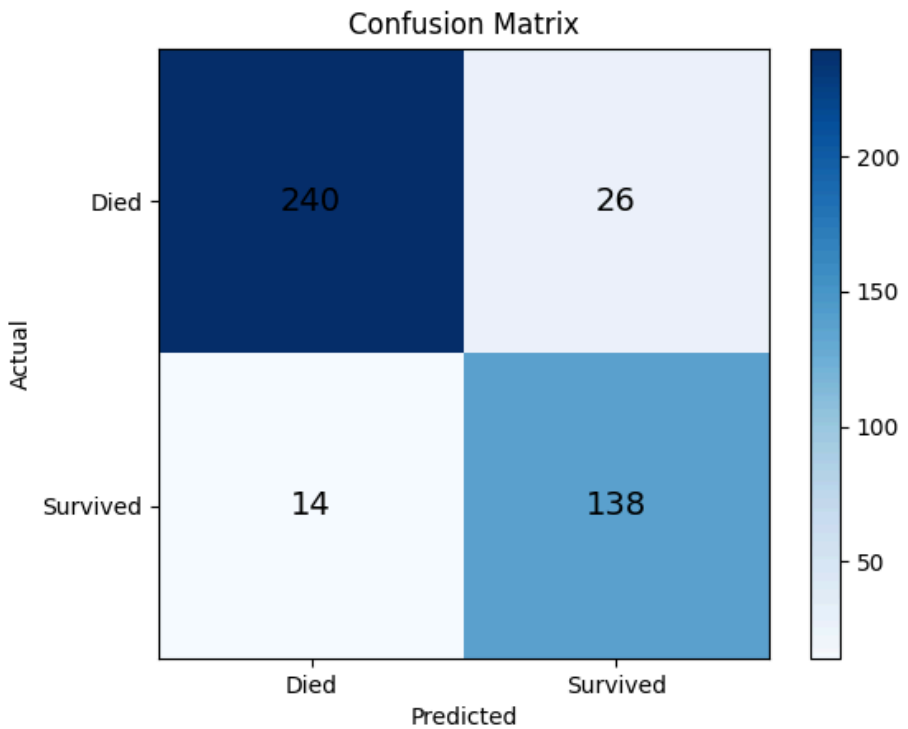
3.4 Single Run Results

The model was trained with: init=He, lr=0.1, epochs=100.

Metric	Value
Train Accuracy (final)	79.4%
Validation Accuracy (final)	79.1%
Test Loss	0.2903
Test Accuracy	90.4%
F1-Score (Died)	0.92
F1-Score (Survived)	0.87


Loss over Epochs
Train Loss
Val Loss
Loss
Epoch
0.75
0.70
0.65
0.60
0.55
0.50
0.45
0
20
40
60
80
100

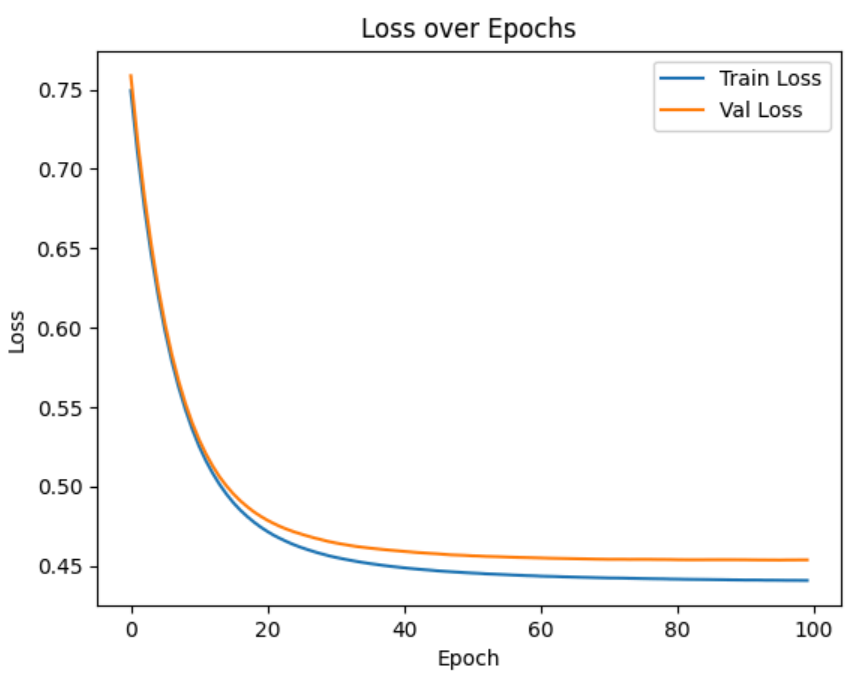

Accuracy over Epochs
Train Acc
Val Acc
Accuracy
Epoch
0.80
0.75
0.70
0.65
0.60
0
20
40
60
80
100

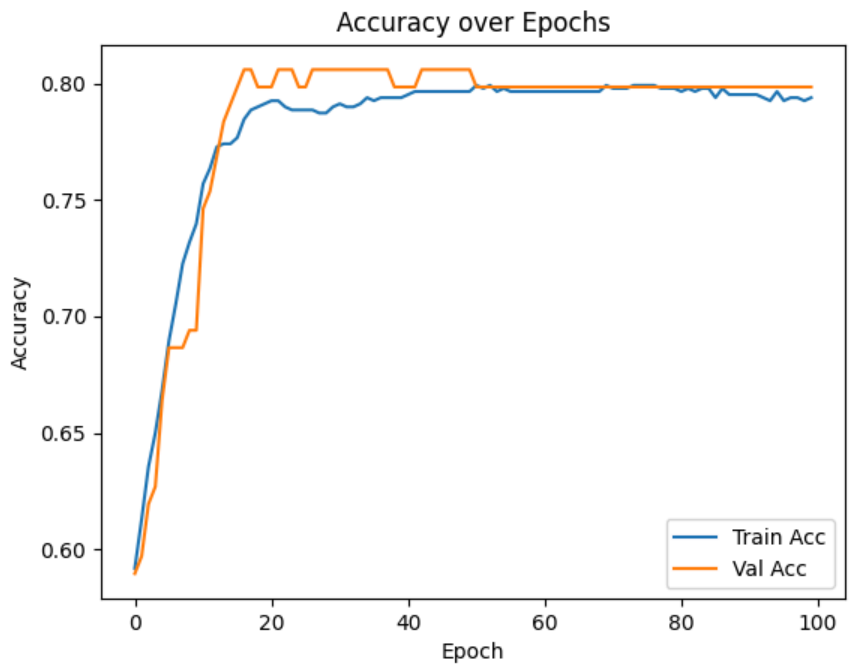

Confusion Matrix
Died
Survived
Actual
240
26
14
138
Died
Survived
Predicted
200
150
100
50

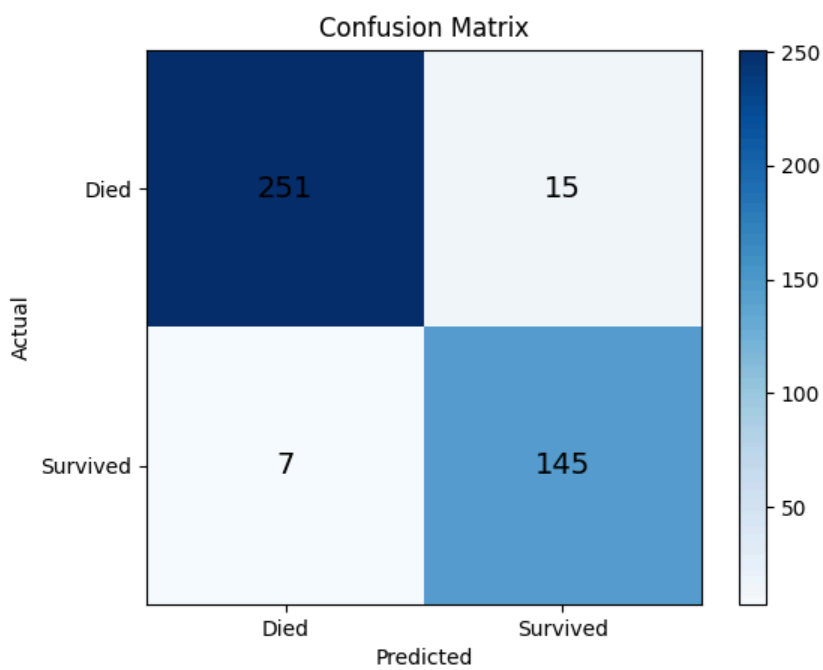
3.5 Experiment Results

#	LR	Epochs	Init	Test Loss	Test Acc	F1 (Died)	F1 (Surv)
1	0.1	100	He	0.2718	92.1%	0.94	0.90
2	0.01	100	He	0.2737	94.7%	0.96	0.93
3	0.5	100	He	0.3080	89.0%	0.91	0.86
4	0.001	100	He	0.4668	79.7%	0.81	0.78
5	0.1	200	He	0.2716	92.6%	0.94	0.90
6	0.1	50	He	0.2695	92.8%	0.94	0.90
7	0.1	100	Random	0.2669	93.3%	0.95	0.91
8	0.1	100	Zeros	0.2691	92.6%	0.94	0.90
9	0.01	200	Random	0.2695	94.3%	0.95	0.92
10	0.5	100	Zeros	0.2714	93.8%	0.95	0.92

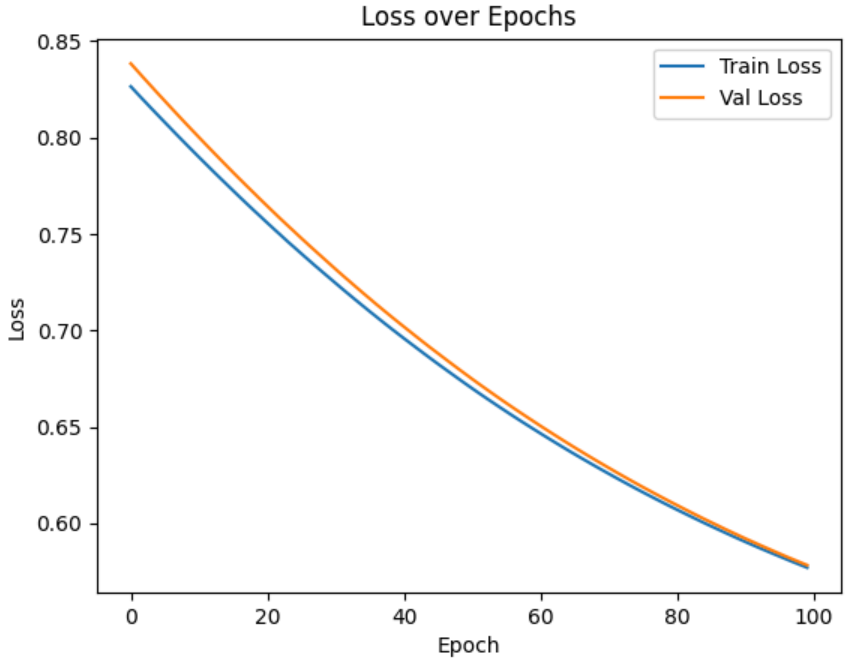
Experiment 2 (Best: lr=0.01, He, 100 epochs - 94.7%)

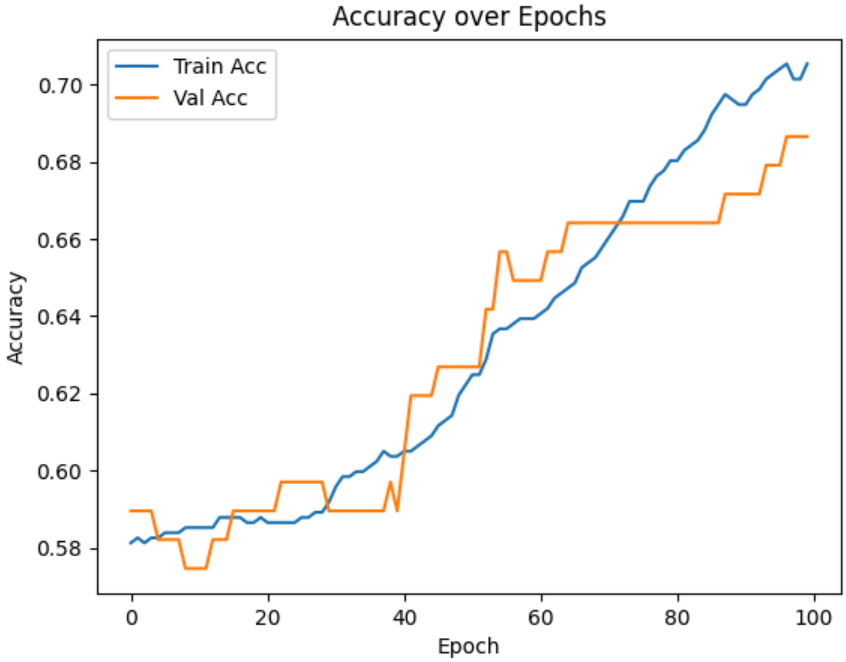




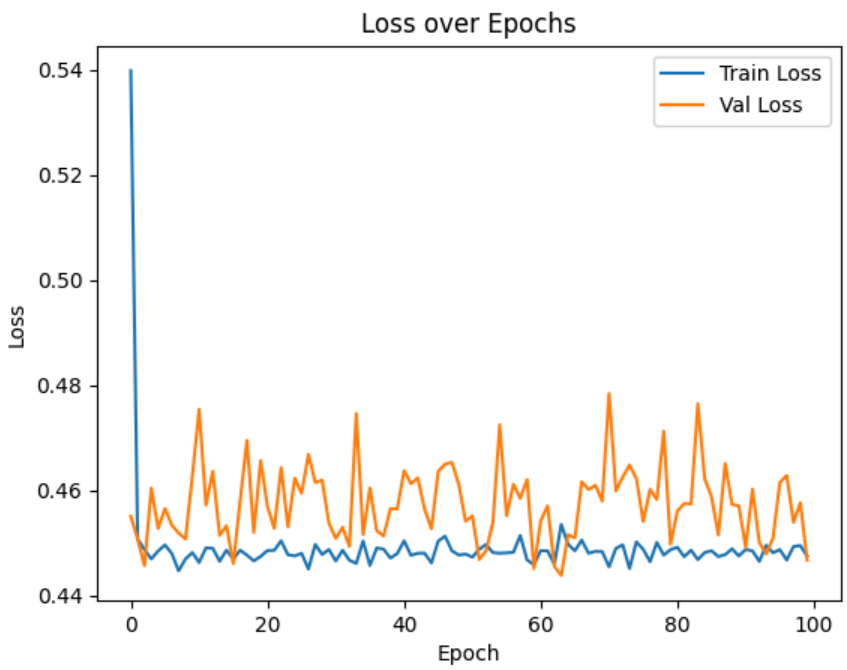


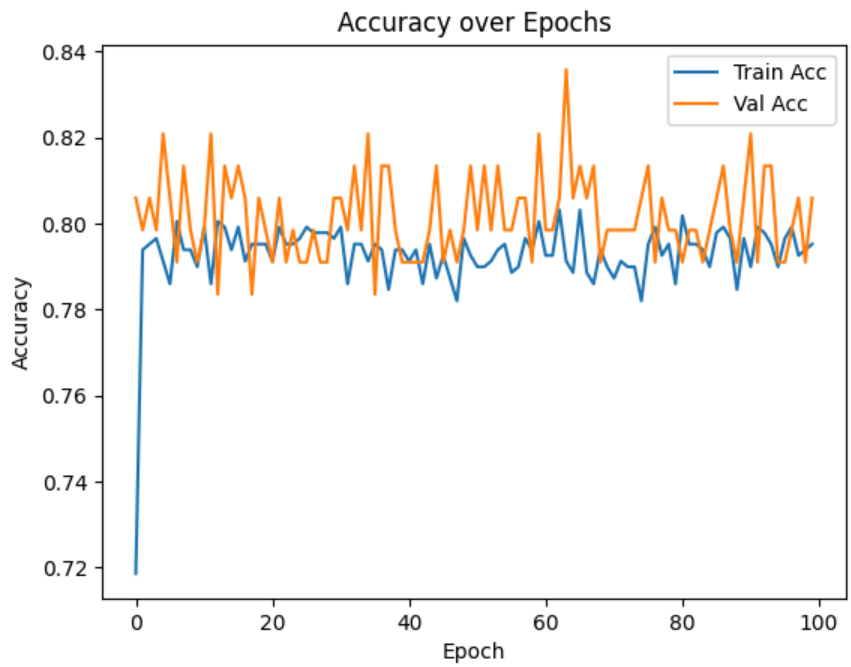
Experiment 4 (lr=0.001 - Slow Learning - 79.7%)

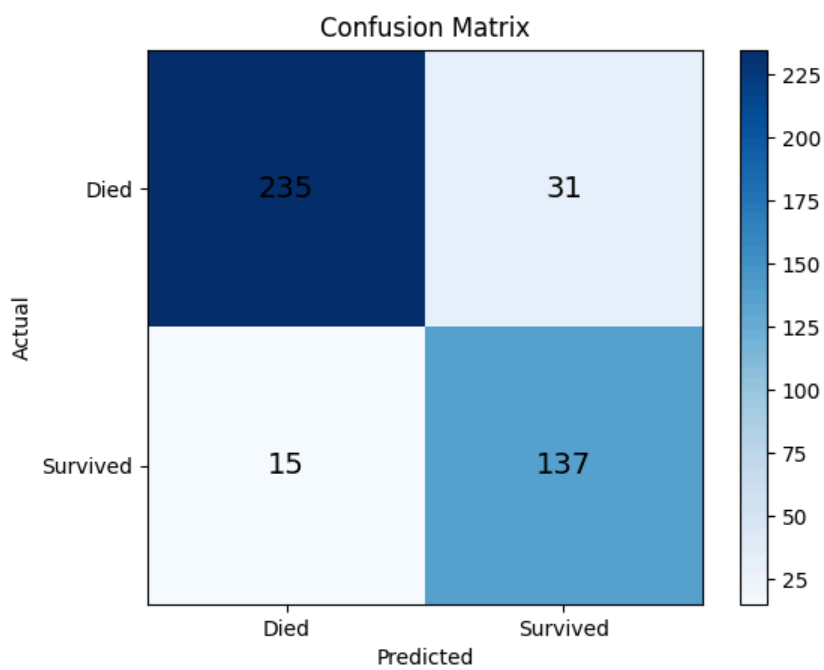

Loss over Epochs
Train Loss
Val Loss
Loss
Epoch
0.85
0.80
0.75
0.70
0.65
0.60
0
20
40
60
80
100


Accuracy over Epochs
Train Acc
Val Acc
Accuracy
Epoch
0.70
0.68
0.66
0.64
0.62
0.60
0.58
0
20
40
60
80
100

Experiment 3 (lr=0.5 - Too Aggressive - 89.0%)




Accuracy over Epochs
Train Acc
Val Acc
Accuracy
Epoch
0.84
0.82
0.80
0.78
0.76
0.74
0.72
0
20
40
60
80
100


Confusion Matrix
Died
Survived
Actual
235
31
15
137
Died
Survived
Predicted
225
200
175
150
125
100
75
50
25

3.6 Analysis

Best Configuration: Experiment 2 (lr=0.01, He init, 100 epochs) achieved 94.7% test accuracy with F1 scores of 0.96 (Died) and 0.93 (Survived).

Learning Rate: lr=0.01 and lr=0.1 both worked well (92-95% accuracy). lr=0.5 was too aggressive, causing fluctuating loss and lower accuracy (89%). lr=0.001 was too slow with only 79.7% accuracy after 100 epochs.

Weight Initialization: Unlike Task 1, initialization type did not significantly affect performance in logistic regression. All three (He, Random, Zeros) achieved 92-93% accuracy. This is because logistic regression has only one layer, so the symmetry breaking problem that affects multi-layer networks does not apply.

Number of Epochs: The model converges quickly. 50, 100, and 200 epochs all gave similar results (92-93%). This indicates that logistic regression on this dataset learns the decision boundary early and doesn't improve much with more training.

All experiments exceeded the 60% accuracy threshold required by the assignment.

4. Conclusion

In this assignment, two machine learning models were implemented from scratch using NumPy:

Task 1 (Linear Regression): A 2-layer neural network was trained on the California Housing dataset. The best configuration achieved Train $R^2 = 0.73$ and Test $R^2 = 0.62$. Key findings: He initialization is important for multi-layer networks, normalization is essential, and learning rate needs to be carefully tuned (0.01 worked best).

Task 2 (Logistic Regression): A single-layer model was trained on the Titanic dataset. The best configuration achieved 94.7% test accuracy. Key findings: logistic regression is less sensitive to initialization, converges quickly, and moderate learning rates (0.01-0.1) work best.

The experiments demonstrate the importance of hyperparameter tuning. Learning rate and normalization had the most significant impact on performance. Weight initialization matters more for deeper networks (Task 1) than shallow ones (Task 2).